

Support Vector Machine (SVM)

Modello

Logica dell'Algoritmo

Un Support Vector Machine è un classificatore che cerca l'**iperpiano ottimale** che separa due classi massimizzando il **margin** (distanza tra l'iperpiano e i punti più vicini di ciascuna classe).

Per dati bidimensionali è una retta, per dati tridimensionali è un piano, per dati p-dimensionali è un **iperpiano** definito da:

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p = 0$$

Dove $\beta = [\beta_1, \dots, \beta_p]$ è il **vettore normale all'iperpiano** (perpendicolare ad esso).

Hard-Margin SVM (Dati Linearmente Separabili)

Nel caso ideale dove i dati sono **perfettamente separabili**, l'obiettivo è trovare i coefficienti $\beta_0, \beta_1, \dots, \beta_p$ che massimizzano il **margin**.

La condizione di separazione è:

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq 1 \quad \forall i$$

Dove $y_i \in \{-1, +1\}$ è l'etichetta della classe.

La distanza tra un punto e l'iperpiano è data da:

$$r_i = \frac{y_i(\beta_0 + \sum_{j=1}^p \beta_j x_{ij})}{||\beta||}$$

Dove $||\beta|| = \sqrt{\sum_{j=1}^p \beta_j^2}$ è la **norma euclidea** del vettore dei coefficienti.

Il **margin** è la distanza minima tra l'iperpiano e il punto di training più vicino:

$$M = \frac{1}{||\beta||}$$

Per massimizzare il margin, è equivalente **minimizzare** $||\beta||$, il che si formula come un problema di ottimizzazione quadratica:

$$\min_{\beta, \beta_0} \frac{1}{2} ||\beta||^2$$

Soggetto al vincolo:

$$y_i \left(\beta_0 + \sum_{j=1}^p \beta_j x_{ij} \right) \geq 1 \quad \forall i = 1, \dots, n$$

Motivazione: perché massimizzare il margin? L'intuizione è che un iperpiano con margin grande è più **robusto** a variazioni nei dati e generalizza meglio su dati non visti.

Soft-Margin SVM (Dati Non Separabili)

Nel caso realistico dove i dati **non sono linearmente separabili**, permettere che alcuni punti violino il margin:

Si introducono **variabili di slack** $\xi_i \geq 0$ che misurano quanto un punto viola il margin:

$$y_i \left(\beta_0 + \sum_{j=1}^p \beta_j x_{ij} \right) \geq 1 - \xi_i \quad \forall i$$

Il problema di ottimizzazione diventa:

$$\min_{\beta, \beta_0, \xi} \left[\frac{1}{2} ||\beta||^2 + C \sum_{i=1}^n \xi_i \right]$$

Dove:

- **Primo termine:** penalità per complessità (piccoli coefficienti = iperpiano semplice)
- **Secondo termine:** penalità per violazioni del margine (ogni violazione costa $C \cdot \xi_i$)
- **Parametro C: iperparametro di regolarizzazione** che controlla il trade-off tra:
 - Margine grande (coefficienti piccoli)
 - Permettere violazioni (perdita di training bassa)

Interpretazione di C:

- **C grande:** il modello penalizza fortemente le violazioni → addattamento ai dati di training (rischio overfitting)
- **C piccolo:** il modello tollera violazioni → priorità al margine grande (rischio underfitting)

Kernel SVM (Trasformazione Non Lineare)

Il limite principale di SVM lineare è che funziona solo se i dati sono (approssimativamente) **linearmente separabili**. Per dati con pattern non lineari, SVM usa il **kernel trick**.

Idea: trasformare i dati in uno spazio di dimensionalità superiore (possibilmente infinita) dove diventano linearmente separabili. Tuttavia, computare esplicitamente la trasformazione è costoso. Il **kernel trick** consente di fare questo implicitamente.

Kernel Trick

Aniché trasformare i dati esplicitamente $\phi(x_i) = [f_1(x_i), f_2(x_i), \dots, f_m(x_i)]$ e poi calcolare prodotti scalari $\phi(x_i)^T \phi(x_j)$, usiamo una **funzione kernel** che calcola direttamente il prodotto scalare nello spazio trasformato:

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

Senza dover esplicitamente calcolare ϕ .

Kernel Comuni

1. Kernel Lineare

$$K(x_i, x_j) = x_i^T x_j$$

Nessuna trasformazione; ritorna alla SVM lineare.

2. Kernel Polinomiale

$$K(x_i, x_j) = (x_i^T x_j + 1)^d$$

Trasforma i dati in uno spazio di polinomi di grado d . Utile per pattern polinomiali.

3. Kernel RBF (Radial Basis Function - Gaussian)

$$K(x_i, x_j) = \exp(-\gamma ||x_i - x_j||^2)$$

Trasforma in uno spazio di dimensionalità infinita. È il kernel **più comunemente usato** perché:

- Molto flessibile, adatto a pattern complessi
- Un solo iperparametro γ (gamma) da tuning
- Buon compromesso tra complessità e performance

Interpretazione di γ :

- γ **grande**: il kernel focalizza su punti molto vicini (può causare overfitting)
- γ **piccolo**: il kernel considera punti lontani (più generale, rischio underfitting)

4. Kernel Sigmoid

$$K(x_i, x_j) = \tanh(\alpha x_i^T x_j + c)$$

Simile al kernel di una rete neurale. Meno comune.

Rappresentazione della Soluzione

La soluzione di SVM è rappresentata come una **combinazione lineare di prodotti kernel**:

$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i y_i K(x, x_i)$$

Dove:

- α_i sono i **coefficienti duali** (non direttamente i β_j)
- Solo una **frazione dei punti di training** ha $\alpha_i > 0$ — questi sono i **Support Vectors**

I support vectors sono i punti più informativi per la classificazione; i punti "facili" (ben separati) hanno $\alpha_i = 0$ e vengono ignorati.

Complessità Computazionale

Training

La complessità di training di SVM dipende dal metodo di ottimizzazione usato:

Hard-Margin SVM (Lineare)

$$O(n^2p) \text{ a } O(n^3p)$$

- Metodo **Quadratic Programming (QP)**: tipicamente $O(n^2p)$ o $O(n^3)$ per il solver
- Con n = numero di campioni, p = numero di feature
- La matrice del kernel è $n \times n$, quindi operazioni quadratiche su essa

Soft-Margin SVM (con variabili di slack)

$$O(n^2p) \text{ a } O(n^3p)$$

Stessa complessità; la penalità C non cambia la complessità asintotica.

Kernel SVM

$$O(n^2p) \text{ a } O(n^3)$$

Per kernel RBF, non è necessario calcolare esplicitamente ϕ (dimensioni potenzialmente infinite):

- Calcolo della matrice kernel: $O(n^2p)$ (comparare ogni coppia di punti)
- Solver QP: $O(n^3)$
- **Total**: $O(n^3)$ (dominato dal solver)

Ottimizzazioni Pratiche

Per dataset **grandi** ($n \gg 10.000$), i solver standard diventano intractabili. Soluzioni:

1. **Stochastic Gradient Descent (SGD) SVM**: $O(n \cdot p)$ ma con più iterazioni
2. **Online SVM**: aggiorna in streaming
3. **Approximate solutions**: vincolamenti per ridurre n
4. **Distributed training**: parallelizzazione su cluster

Inference (Previsione)

$$O(m \cdot p)$$

Dove m è il **numero di support vectors** ($m \leq n$).

La previsione richiede:

$$f(x_{\text{new}}) = \beta_0 + \sum_{i \in SV} \alpha_i y_i K(x_{\text{new}}, x_i)$$

Se il modello ha molti support vectors ($m \approx n$), inference diventa lenta. Se $m \ll n$ (case ideale), inference è veloce.

Memoria

$$O(n^2 + m \cdot p)$$

- Matrice kernel: $O(n^2)$
- Coefficienti: $O(m)$ (support vectors) + $O(p)$ (per ogni kernel non lineare)

Per dataset molto grandi, la memoria della matrice kernel può essere **proibitiva**.

Scalabilità: Conclusioni

- **Training**: cubic in n — **critico per dataset grandi**
 - Per $n > 100.000$, impraticabile senza approssimazioni
 - Per $n < 10.000$, ragionevole
- **Inference**: dipende da m (numero support vectors)
 - Se m è piccolo: veloce
 - Se $m \approx n$: lento
- **Memoria**: proibitiva per dataset enormi ($n > 1M$)

Rappresentazione Interna

Struttura del Modello

La rappresentazione interna di SVM è:

Support Vectors: $[x_1, x_2, \dots, x_m]$ (sottinsieme di n punti di training)

Coefficienti: $[\alpha_1, \alpha_2, \dots, \alpha_m]$ e β_0

Kernel: $K(\cdot, \cdot)$ (funzione)

Previsione: $f(x) = \beta_0 + \sum_i \alpha_i y_i K(x, x_i)$

Implicazioni per la Spiegabilità

Contro:

- La previsione dipende da un **subset arbitrario** di punti di training (support vectors)
- Non è immediatamente chiaro **perché** un particolare punto è un support vector

- Per kernel non lineari (es. RBF), la trasformazione ϕ è implicita e non visualizzabile (dimensioni infinite)
- L'interpretazione di α_i non è intuitiva: non corrisponde direttamente a "importanza" della feature

Pro:

- Almeno i **support vectors sono identità**: puoi elencare quali punti di training hanno influenzato la previsione
- Questo è meglio di una rete neurale dove la decisione è completamente dispersa nei parametri
- Puoi analizzare i support vectors localmente per capire il ragionamento

Contrasto con altri modelli:

- **LR/LogR**: coefficienti sono interpretabili per feature (spiegabilità per feature)
- **DT**: cammino è tracciabile (spiegabilità per logica)
- **SVM**: support vectors sono tracciabili (spiegabilità per precedenti)

Vincoli sui Dati

Linearità (Hard-Margin SVM)

Hard-margin SVM **richiede separabilità lineare esatta**. Se i dati non sono linearmente separabili, il problema di ottimizzazione **non ha soluzione fattibile**.

Soluzione: Soft-margin SVM, che tollera violazioni.

Linearità (Soft-Margin SVM)

Soft-margin SVM è ancora lineare nello spazio dei dati (senza kernel). Se i dati hanno pattern non lineari, si usa kernel SVM.

Scalabilità

Come discusso, **training è $O(n^3)$** , quindi SVM non scala bene a dataset enormi senza approssimazioni.

Bilanciamento delle Classi

SVM può essere influenzato da **classi sbilanciate**:

- L'algoritmo massimizza il margine complessivo, il che potrebbe favorire la classe maggioritaria
- Se una classe è il 95% e l'altra il 5%, il margine "massimo" potrebbe consistere nel classificare tutto come la classe maggioritaria

Soluzioni:

- **Pesi di classe**: assegnare peso inverso alla frequenza (classe rara = peso alto)
- **Tuning di C**: aumentare C per la classe minoritaria
- **Ricampionamento**: oversampling della classe rara, undersampling della maggioritaria

Normalizzazione delle Feature

SVM usa la **distanza euclidea** ($\|x_i - x_j\|$) nel kernel, quindi è **sensibile alla scala**:

- Se una feature va da 0-1 e un'altra da 0-1.000.000, quest'ultima domina
- **Pratica standard**: normalizzare tutte le feature (es. standardizzazione z-score o min-max scaling)

Nessun'altra Assunzione Strutturale

A differenza di LR, SVM **non ha assunzioni** su:

- Normalità delle distribuzioni
- Omoschedasticità

- Relazioni lineari (soprattutto con kernel)

Capacità Predittive

Punti di Forza

1. Eccellente per Pattern Non Lineari (con Kernel)

- Kernel RBF può approssimare funzioni arbitrariamente complesse
- Spesso outperform modelli lineari su dati reali

2. Robusto a Outlier

- Soft-margin SVM usa slack variables; outlier hanno impatto limitato
- Il margine è costruito basandosi su support vectors, non su tutti i punti

3. Funziona Bene in Alte Dimensioni

- Efficace anche quando $p > n$ (numero di feature > numero di campioni)
- Modelli lineari tendono a overfitting in questi scenari

4. Principi Teorici Solidi

- Massimizzazione del margine ha fondamenti teorici (theory of VC dimension, generalization bounds)
- Garantisce che la soluzione è **globalmente ottimale** (problema convesso)

5. Versatile

- Funziona sia per classificazione che per regressione (SVR)
- Multi-classe mediante one-vs-rest o one-vs-one

Punti di Debolezza

1. Scarsa Interpretabilità

- Specialmente con kernel non lineari, difficile spiegare il ragionamento

2. Sensibilità a Iperparametri

- Scelta di C (soft-margin) e γ (RBF kernel) critica
- Tuning male = performance disastrose

3. Inefficienza su Dataset Grandi

- Complessità training $O(n^3)$ — impraticabile per $n \gg 100.000$
- Anche inference può essere lenta se molti support vectors

4. Non Probabilistico

- Output è una classificazione hard (classe -1 o $+1$)
- Per ottenere probabilità, è necessario post-processing (Platt scaling)

5. Multicollinearità Non Gestita

- Se feature sono fortemente correlate, performance può deteriorarsi
- Richiede feature selection o regolarizzazione

Metriche per la Confidenza

Metriche di Classificazione Standard

Stesse metriche di LogR e DT:

- **Confusion Matrix:** TP, TN, FP, FN
- **Accuracy:** $(TP + TN) / (TP + TN + FP + FN)$
- **Precision:** $TP / (TP + FP)$
- **Recall/Sensitivity:** $TP / (TP + FN)$
- **Specificity:** $TN / (TN + FP)$
- **F1-Score:** media armonica di Precision e Recall
- **ROC Curve e AUC:** trade-off tra TPR e FPR

Distanza dall'Iperpiano

Misura della **confidenza specifica di SVM**:

$$d_i = y_i \left(\beta_0 + \sum_{j=1}^p \beta_j x_{ij} \right)$$

Questa è la **distanza (segnata) del punto x_i dall'iperpiano**, normalizzata da $||\beta||$:

$$\text{Distanza Normalizzata} = \frac{d_i}{||\beta||}$$

Interpretazione:

- $d_i > 1$: punto ben classificato, lontano dall'iperpiano (confidenza alta)
- $d_i \approx 1$: punto sul margine (confine della decisione)
- $0 < d_i < 1$: punto nella regione di slack (violazione del margine)
- $d_i < 0$: punto classificato erroneamente

Questa distanza è un **proxy per la confidenza della previsione**: punti lontani dall'iperpiano sono predetti con confidenza maggiore.

Platt Scaling (Probabilità Posteriori)

SVM non produce probabilità di default. Per ottenere stime di probabilità $P(y = 1 | x)$, si usa **Platt scaling**:

$$P(y = 1) = \frac{1}{1 + \exp(A \cdot d + B)}$$

Dove A, B sono parametri appresi su un validation set, calibrando la distanza dell'iperpiano a probabilità.

Limite: aggiunge complessità e calibrazione; le probabilità risultanti non hanno la stessa garanzia teorica della regressione logistica.

Metriche per la Comprensione e Spiegabilità

1. Identificazione dei Support Vectors

Il set dei support vectors è la **feature più interpretabile** di SVM:

Support Vectors: [istanza_3, istanza_15, istanza_42, ...]

Rappresentano i punti di training che hanno **influenzato veramente la decisione**. Punti ben separati hanno $\alpha_i = 0$ e sono ignorati.

Utilizzo pratico:

- Analizzare i support vectors per capire quali punti sono "difficili" (contorno decisionale)
- Visualizzare support vectors vs punti di background per ispezionare il modello

- Se molti support vectors = modello complesso; se pochi = modello semplice

2. Pesi dei Support Vectors (α_i)

I coefficienti $\alpha_i > 0$ associati ai support vectors indicano il loro "peso" nella decisione:

$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i y_i K(x, x_i)$$

Interpretazione:

- α_i grande = il support vector x_i ha grande influenza
- α_i piccolo = il support vector x_i ha piccola influenza

Caveat: interpretazione limitata per kernel non lineari, perché non è immediatamente chiaro cosa stia facendo il kernel.

3. Feature Importance (via Permutation o SHAP)

SVM non produce naturalmente feature importance. Soluzioni:

Permutation Importance:

- Permutare casualmente una feature nei dati di test
- Misurare il degrado in performance
- Grandi degni = feature importante

SHAP (SHapley Additive exPlanations):

- Assegnare ogni output tra le feature usando valori Shapley
- Applicabile a qualsiasi modello (SVM, NN, etc.)
- Computazionalmente costoso pero produce spiegazioni teoricamente fondate

4. Visualizzazione della Separazione (2D/3D)

Per dataset a bassa dimensionalità, visualizzare:

- Punti di training e loro classi
- Iperpiano di decisione (retta in 2D, piano in 3D)
- Margine (linee parallele a distanza $M = 1 / ||\beta||$)
- Support vectors (segnati diversamente)

Limite: per $p > 3$, non è possibile visualizzare direttamente. Usare proiezione PCA per ridurre dimensioni e visualizzare.

5. Analisi Locale: Contrastive Explanation

Per una previsione di istanza x_{new} :

- Trovare i support vectors più vicini (usando la metrica del kernel)
- Mostrare come differiscono da x_{new}

Esempio:

Istanza X è classificata come Positiva perché simile al Support Vector SV_5 (che è Positivo), diversa dal Support Vector SV_12 (che è Negativo).

Differenze principali:

- X.Income > SV_5.Income (simile)
- X.Age < SV_12.Age (dissimile)

Limiti di Predizione

Non Probabilistico per Default

SVM output una classificazione hard (+1 o -1), non una probabilità:

- Non ottenere stime naturali di incertezza
- Platt scaling produce probabilità, ma richiede calibrazione aggiuntiva

Conseguenza: per problemi dove la probabilità importa (es. medicina), SVM deve essere post-processato.

Sensibilità al Tuning di Iperparametri

Performance di SVM è **molto sensibile** a:

- **C (soft-margin):** piccolo C = underfitting; grande C = overfitting
- **γ (RBF kernel):** piccolo γ = generale; grande γ = specifico (overfitting)

Conseguenza: il tuning di iperparametri (grid search, random search, Bayesian optimization) è **mandatorio**, il che aggiunge complessità.

Sensibilità alla Normalizzazione delle Feature

Senza normalizzazione, feature con scale grandi dominano. SVM è **molto sensibile** a questo.

Mitigazione: normalizzare sempre le feature (standardizzazione z-score è standard).

Performance su Dataset Sbilanciate

Come discusso, SVM tende a favorire la classe maggioritaria.

Mitigazione: usare class weights (inversamente proporzionali alla frequenza).

Inefficienza su Dataset Grandi

Complessità $O(n^3)$ rende SVM **impraticabile per $n \gg 100.000$** .

Mitigazioni possibili:

- Usare approssimazioni (Nyström approximation, stochastic gradient descent)
- Sottocampionare per training, validare su full dataset
- Usare alternative scalabili (logistic regression, gradient boosting)

Limiti di Spiegabilità

Opacità della Trasformazione Non Lineare (Kernel)

Con kernel RBF, i dati sono trasformati in uno **spazio di dimensionalità infinita**:

$$K(x, x') = \exp(-\gamma ||x - x'||^2)$$

Questa trasformazione $\phi(x)$ è **implicita e non visualizzabile**. Conseguenze:

- Non puoi interpretare cosa stia facendo il modello nello spazio trasformato
- L'effetto delle feature sulla previsione è **nascosto dietro il kernel**
- Spiegare una previsione richiede capire il kernel, che è non intuitivo per i non-esperti

Interpretazione Limitata di α_i

I pesi α_i dei support vectors non hanno interpretazione diretta:

- Non significano "questa feature è importante"
- Non significano "questo support vector cambia la probabilità del 10%"
- Sono artefatti dell'ottimizzazione quadratica, non quantità interpretabili

Dipendenza da Support Vectors Arbitrari

Quali punti diventano support vectors dipende dalla **geometria dello spazio e dall'iperparametro C**:

- Cambiar $C \rightarrow$ diversi support vectors
- Cambiar dataset leggermente $\rightarrow$ diversi support vectors
- Non è evidente **perché** un punto è un support vector (è dovuto alla posizione geometrica, ma questa non è facilmente comunicabile)

Scarsa Tracciabilità Globale

Mentre puoi identificare i support vectors, **tracciare il ragionamento globale è difficile**:

- Non puoi dire "la feature X è il driver principale della classificazione" (come in LR)
- Non puoi tracciare una sequenza logica (come in DT)
- Rimane una "scatola nera" anche dopo aver visto i support vectors

Nessuna Spiegazione Naturale per Feature Importanza

A differenza di:

- **LR**: coefficienti diretti
- **DT**: feature importance dai split

SVM non produce feature importance naturale. Devi usare tecniche esterne (SHAP, permutation importance).

Confronto con altri Algoritmi

Vs. Logistic Regression

Aspetto	LogR	SVM
Output	Probabilità	Classe (distanza)
Interpretabilità	★★★★ Alta	★★ Bassa
Pattern Non-Lineare	★ (no kernel)	★★★★★ (con kernel RBF)
Scalabilità	★★★★ $O(n \cdot p \cdot k)$	★★ $O(n^3)$
Teoria	Probabilistica	Geometrica (margine)
Quando usare	Interpretabilità critica	Pattern non-lineare complesso

Vs. Decision Tree

Aspetto	DT	SVM
Interpretabilità	★★★★★ Altissima	★★ Bassa
Pattern Non-Lineare	★★★★★ Naturale	★★★★★ Con kernel
Scalabilità	★★★★ $O(n \cdot p \cdot \log n)$	★★ $O(n^3)$
Robustezza Outlier	★★ (sensibile)	★★★★★ (robusto)
Stabilità	★★ (instabile)	★★★★★ (stabile)
Quando usare	Massima interpretabilità	Pattern complessi + stabilità

Vs. Neural Networks

Aspetto	NN	SVM
Capacità	★★★★★ Illimitata	★★★★ Molto buona
Interpretabilità	★ Scatola nera	★★ Leggermente migliore
Scalabilità	★★★★★ Eccellente	★★ Scarsa
Data Requirement	★ (molto dati)	★★★★ (meno dati)
Training	Lungo, GPU-friendly	Veloce ma $O(n^3)$
Teoria	Inesistente	Solida
Quando usare	Big data + capacità illimitata	Data medio + interpretabilità

Vs. Random Forest / Gradient Boosting

Aspetto	Ensemble	SVM
Performance	★★★★★ Spesso migliore	★★★★ Buona
Interpretabilità	★★ (migliore di NN)	★★ Simile
Scalabilità	★★★★ (meglio di SVM)	★★
Robustezza Iperparametri	★★★★ Meno sensibile	★★ Molto sensibile
Implementazione	Easy (sklearn)	Easy (sklearn)
Quando usare	Predizione è priorità #1	Stabilità, spiegabilità, dati piccoli

Varianti e Estensioni

1. Multi-Class SVM

SVM è originariamente binario. Per multi-classe:

One-vs-Rest (OvR):

- Creare K classificatori binari (K = numero di classi)
- Ogni classificatore separa una classe dal resto
- Classe finale = massima confidenza

One-vs-One (OvO):

- Creare $K(K-1)/2$ classificatori binari (uno per coppia di classi)
- Voting: conta quante volte ogni classe è predetta
- Più computazionalmente costoso, spesso più accurato

2. Support Vector Regression (SVR)

Estensione di SVM per **regressione** (output continuo):

- Aniché maximizzare il margine di classificazione, minimizzare l'errore di predizione
- Introduce un'**epsilon-insensitive loss**: errori piccoli $< \epsilon$ sono tollerati
- Utile per predizione continua con gestione robusta di outlier

3. Uno-Classe SVM (One-Class SVM)

Per problemi di **anomaly detection** / **outlier detection**:

- Imparare un "confine" intorno ai dati normali
- Punti fuori da questo confine sono anomalie

- Usa soft-margin con una frazione ν di campioni come support vectors

4. Relevance Vector Machine (RVM)

Variante Bayesiana di SVM:

- Produce probabilità di output come LogR
- Spesso meno support vectors (più sparso)
- Trade-off: meno efficienza di SVM, più interpretabilità

Raccomandazioni Pratiche

Quando Usare SVM

✓ Ottime scelte:

- Dataset di piccolo-medio volume ($n < 100.000$)
- Pattern non lineari complessi
- Necessità di stabilità e robustezza
- Dati ad alta dimensionalità ($p > n$)
- Outlier presenti nel dataset

✗ Cattive scelte:

- Dataset molto grandi ($n > 1.000.000$)
- Interpretabilità è critica per decisioni (es. medicina, finanza regolata)
- Tuning di iperparametri non è possibile
- Output probabilistico naturale necessario

Tuning Consigliate

1. **Normalizzare sempre le feature** (z-score o min-max)
2. **Griglia di ricerca per C e γ :**
 - C: [0.1, 1, 10, 100, 1000]
 - γ : [0.001, 0.01, 0.1, 1, 'auto']
3. **Cross-validation:** 5-10 fold per valutare
4. **Class weights:** se classi sbilanciate, usare pesi inversi
5. **Kernel:** iniziare con RBF; provare polinomiale se pattern sospetto

Prompt